

Kids Routine — Chuleta Defensa TFG

DAW · Marcos Pérez · 2026

URL en producción: `https://kids-routine-production.up.railway.app`
Demo: `demo@kidsroutine.com` / `demo123` · **Sofía PIN:** `1234` · **Pablo PIN:** `5678`

1. ¿Qué es Kids Routine?

Aplicación web para que los padres gestionen las rutinas y tareas diarias de sus hijos. Los niños completan tareas, acumulan monedas virtuales y las canjean por recompensas reales que define el padre.

Problema que resuelve

- Padres que no tienen forma digital de hacer seguimiento de tareas del hogar de sus hijos.
- Niños que necesitan motivación tangible: sistema de recompensas gamificado.
- Acceso infantil seguro sin email/contraseña: solo PIN numérico de 4 dígitos.

Flujo principal



2. Stack tecnológico

Capa	Tecnología	Versión	Por qué
Backend	Laravel (PHP)	13.0 / PHP 8.3	Framework MVC robusto, ORM Eloquent, autenticación integrada
Base de datos	MySQL	8.x	Relacional, ideal para datos con muchas relaciones (tareas, hijos, canjes)
Frontend	Blade + Tailwind CSS	Tailwind 4.0	Motor de plantillas de Laravel, CSS utility-first sin JavaScript pesado
Build tool	Vite	8.0	Compilador de assets rápido, integrado con Laravel
Servidor local	Laragon	—	Entorno Windows all-in-one (Apache, PHP, MySQL)
Despliegue	Railway	—	PaaS gratuito, desplegado desde GitHub automáticamente
Control de versiones	Git + GitHub	—	Repositorio: mmarcospperez/kids-routine

3. Arquitectura MVC

Laravel sigue el patrón **Modelo-Vista-Controlador**:

Capa	Ubicación	Responsabilidad
Modelos	app/Models/	Representan las tablas BD + relaciones + lógica de negocio
Controladores	app/Http/Controllers/	Reciben la petición HTTP, llaman al modelo, devuelven la vista
Vistas	resources/views/	HTML con Blade (plantillas PHP de Laravel)
Rutas	routes/web.php	Mapear URL → controlador
Middlewares	app/Http/Middleware/	Filtros que se ejecutan antes del controlador (autenticación)
Migraciones	database/migrations/	Definen la estructura de la BD en código PHP

Estructura de carpetas relevante

```
kids_routine/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Auth/           → Login, Registro
│   │   │   ├── Padre/        → Dashboard, Hijo, Tarea, Recompensa, Validacion, Canje, Estadis
│   │   │   └── Hijo/         → Sesión (PIN), Dashboard hijo
│   │   └── Middleware/
│   │       ├── EsPadre.php    → Comprueba que hay sesión de padre
│   │       └── EsHijo.php     → Comprueba que hay sesión de hijo en session()
│   ├── Models/
│   ├── Services/
│   │   └── AchievementService.php → Rachas, logros y niveles (centraliza toda la gamificación)
│   └── Models/
│       ├── User.php           → tabla: usuarios
│       ├── Hijo.php           → tabla: hijos
│       ├── Tarea.php          → tabla: tareas
│       ├── TareaInstancia.php → tabla: tarea_instancias
│       ├── Recompensa.php     → tabla: recompensas
│       ├── Canje.php          → tabla: canjes
│       └── HistorialMonedas.php → tabla: historial_monedas
├── database/
│   ├── migrations/           → 8 migraciones (una por tabla)
│   └── seeders/DatabaseSeeder.php → datos de demo
├── resources/views/          → plantillas Blade
├── routes/web.php             → 39 rutas
├── bootstrap/app.php          → registro de middlewares
├── railway.toml               → configuración despliegue
└── nixpacks.toml              → build en Railway
```

4. Base de datos

Diagrama de entidades

Tabla	PK	Campos clave	Relaciones
-------	----	--------------	------------

usuarios	id_usuario	nombre, email, password_hash, rol	1 padre → N hijos, N recompensas
hijos	id_hijo	nombre, edad, pin_hash, monedas, monedas_tope, intentos_fallidos, bloqueado_hasta	N hijos → 1 padre; 1 hijo → N tareas, canjes, historial
tareas	id_tarea	titulo, monedas_recompensa, tipo, recurrence (DIARIA/SEMANAL/PERSONALIZADA), dias_semana	1 tarea → N instancias
tarea_instancias	id_instancia	fecha_programada, estado (PENDIENTE/COMPLETADA/VALIDADA/RECHAZADA/CADUCADA), fecha_completada, comentario_padre	pertenece a tarea + hijo
recompensas	id_recompensa	nombre, monedas_necesarias, activa	pertenece a padre; 1 recompensa → N canjes
canjes	id_canje	monedas_gastadas, estado (PENDIENTE/APROBADO/ENTREGADO/RECHAZADO), fecha_solicitud	pertenece a hijo + recompensa
historial_monedas	id_historia	cantidad, saldo_anterior, saldo_posterior, motivo	pertenece a hijo (auditoría de cada movimiento)
auditoria	id_auditoria	tabla, accion, datos anteriores/nuevos	log general del sistema
configuracion_juegos	id	id_padre, juego (varchar 50), monedas_por_partida (default 5), activo (boolean)	Configuración del padre para cada juego. UNIQUE (id_padre, juego)
partidas	id	id_hijo, juego, monedas_ganadas, created_at	Historial de partidas ganadas; se filtra por fecha para el límite diario
logros	id	clave (unique), nombre, descripcion, icono, monedas_bonus	Catálogo de logros desbloqueables (primera tarea, 5/25/100 tareas, primera partida, primer canje...)
hijo_logros	id_hijo, id_logro	obtenido_at	Tabla pivot: registra qué logros ha obtenido cada hijo y cuándo

Columnas añadidas en la megamigración:

- **hijos**: **racha_actual** INT, **racha_max** INT, **monedas_historicas** INT (acumulador que nunca baja, para calcular nivel)

- `tareas` : `franja` ENUM(`CUALQUIERA` , `MANANA` , `TARDE` , `NOCHE`), `categoria` VARCHAR(50)

Nota: el valor se llama `MANANA` (sin tilde) en el ENUM para evitar problemas de charset en MySQL en Railway.

Decisión de diseño: Las tablas usan nombres en español (`usuarios` , `hijos` ...) en vez de los convencionales de Laravel (`users`). Por eso los modelos definen explícitamente `$table` y `$primaryKey` . También se desactivan los timestamps automáticos con `public $timestamps = false` y se gestiona `fecha_creacion` manualmente donde hace falta.

5. Modelos — detalles clave

User.php — el padre

```
protected $table = 'usuarios';
protected $primaryKey = 'id_usuario';
public $timestamps = false;

// Laravel busca getAuthPassword() para bcrypt — aquí apunta a password_hash
public function getAuthPassword(): string { return $this->password_hash; }

// Relaciones
public function hijos() { return $this->hasMany(Hijo::class, 'id_padre', 'id_usuario'); }
public function recompensas() { return $this->hasMany(Recompensa::class, 'id_padre', 'id_usuario');
```

Hijo.php — el niño

```
// Bloqueo temporal por intentos fallidos de PIN
public function estaBloqueado(): bool {
    return $this->bloqueado_hasta !== null && now()->lt($this->bloqueado_hasta);
}

// Foto de perfil: devuelve URL pública o null si no tiene foto
public function avatarUrl(): ?string {
    return $this->avatar ? Storage::url($this->avatar) : null;
}

// Color de fondo determinista cuando no hay foto (basado en id_hijo)
public function avatarColor(): string {
    $colores = ['linear-gradient(135deg,#6366f1,#8b5cf6)',
                'linear-gradient(135deg,#ec4899,#f97316)', ...];
    return $colores[$this->id_hijo % count($colores)];
}
```

En la vista se usa el patrón: `@if($hijo->avatarUrl())  @else <div style="background:{{ $hijo->avatarColor() }}">inicial</div> @endif`

TareaInstancia.php — ciclo de vida de una tarea

Cada "ocurrencia" de una tarea es una instancia independiente con su propio estado:

Estado	Quién lo cambia	Significado
PENDIENTE	Sistema (al crear)	Tarea asignada para hoy, sin completar
COMPLETADA	El hijo	El niño dice que la hizo, espera validación del padre
VALIDADA	El padre	El padre confirma → se suman monedas al hijo
RECHAZADA	El padre	El padre rechaza (puede poner comentario)
CADUCADA	Sistema	Pasó el día sin completarse

6. Autenticación dual

Padre — email + contraseña (Laravel Auth)

```
// LoginController.php
Auth::attempt(['email' => $email, 'password' => $password])
// Laravel llama internamente a getAuthPassword() del modelo User
// → devuelve $this->password_hash → bcrypt check automático
```

Hijo — PIN de 4 dígitos

```
// SesionController.php
$hijo = Hijo::find($request->id_hijo);

if ($hijo->estaBloqueado()) → error "bloqueado X minutos"

if (Hash::check($pin, $hijo->pin_hash)) {
    session(['hijo_id' => $hijo->id_hijo]); // guarda en sesión PHP
    return redirect()->route('hijo.dashboard');
} else {
    $hijo->increment('intentos_fallidos');
    if ($hijo->intentos_fallidos >= 3) {
        $hijo->update(['bloqueado_hasta' => now()->addMinutes(15)]);
    }
}
```

Middlewares de protección

Middleware	Alias	Comprueba	Redirige si falla
EsPadre	padre	Auth::check()	/login
EsHijo	hijo	session()->has('hijo_id')	/hijo/seleccionar

Doble capa en la sesión del hijo: El hijo usa la sesión PHP del padre (que está logueado) pero añade `hijo_id` a esa sesión. Así no necesita su propia cuenta de usuario — solo el padre tiene cuenta.

7. Rutas (web.php)

Prefijo	Middleware	Rutas principales
/	ninguno	GET / (welcome), GET /login, POST /login, GET /registro, POST /registro, POST /logout
/padre	padre	GET /dashboard CRUD /hijos (index, create, store, edit, update, destroy) /tareas (index, create, store, destroy) /recompensas (index, create, store, destroy) GET /validaciones, POST /instancias/{id}/validar, /rechazar GET /canjes, POST /canjes/{id}/aprobar, /rechazar, /entregar
/hijo	padre	GET /seleccionar, POST /pin, POST /verificar-pin, POST /salir
/hijo	padre + hijo	GET /dashboard, POST /tareas/{id}/completar, GET /recompensas, POST /recompensas/{id}/canjear GET /juegos, GET /juegos/{juego}, POST /juegos/{juego}/completar
/padre/juegos	padre	GET /juegos (configuración), POST /juegos (guardar monedas y activo por juego)

Total: ~45 rutas. Todas nombradas (ej: `padre.hijos.store` , `hijo.juegos.completar`) para usar `route()` en vistas.

8. Vistas (Blade)

Layouts (plantillas base)

Layout	Usado por	Contiene
layouts/guest.blade.php	Login, Registro	Fondo simple, sin navbar
layouts/app.blade.php	Panel padre	Navbar, menú lateral, nombre usuario
layouts/hijo.blade.php	Panel hijo	Diseño colorido infantil, nombre hijo, monedas

Árbol de vistas

```
resources/views/  
├─ layouts/           → guest, app, hijo  
├─ components/  
│   └─ moneda.blade.php      → SVG de moneda reutilizable (<x-moneda />)  
├─ auth/              → login, register  
├─ padre/  
│   ├── dashboard.blade.php  → resumen general del padre  
│   ├── hijos/              → index, create, edit  
│   ├── tareas/             → index, create  
│   ├── recompensas/        → index, create  
│   ├── validaciones.blade.php → lista de tareas pendientes de validar  
│   ├── canjes.blade.php     → lista de solicitudes de recompensa  
│   ├── estadisticas.blade.php → dashboard: tasas, gráfico semanal, juegos, calendario  
│   └─ juegos/index.blade.php → configuración de juegos (monedas + activo)  
├─ hijo/  
│   ├── seleccionar.blade.php → elige qué hijo entra  
│   ├── pin.blade.php         → introduce PIN  
│   ├── dashboard.blade.php  → tareas de hoy + monedas  
│   └─ recompensas.blade.php → tienda de recompensas
```

	└─ juegos/	
	└─ index.blade.php	→ lista de 5 juegos con estado del día
	└─ sumas.blade.php	→ Operaciones Matemáticas (JS)
	└─ memoria.blade.php	→ Juego de Memoria (JS)
	└─ ahorcado.blade.php	→ Adivina la Palabra (JS)
	└─ ordenar.blade.php	→ Ordena los Números (JS)
	└─ quiz.blade.php	→ Quiz de Conocimiento (JS)
	└─ welcome.blade.php	→ página de inicio pública

Blade — sintaxis básica

```

@extends('layouts.app')           ← hereda layout
@section('content') ... @endsection ← define sección

{{ $variable }}                   ← imprimir con escape HTML (previene XSS)
{!! $html !!}                     ← imprimir sin escape (HTML raw – usar con cuidado)

@foreach ($hijos as $hijo) ... @endforeach
@if ($hijo->estaBloqueado()) ... @endif
route('padre.hijos.store')        ← genera la URL de la ruta nombrada

<x-moneda />                      ← Blade Component: renderiza components/moneda.blade.php

```


9. Sistema de monedas

Las monedas son el núcleo gamificado de la app. Flujo completo:

Acción	Efecto en monedas	Registro en historial_monedas
Padre valida tarea	<code>hijo.monedas += tarea.monedas_recompensa</code>	Sí — motivo "TAREA_VALIDADA"
Padre rechaza tarea	Sin cambio	No
Hijo solicita canje	<code>hijo.monedas -= recompensa.monedas_necesarias</code> (en reserva hasta resolución)	Sí — motivo "CANJE_SOLICITADO"
Padre rechaza canje	Se devuelven las monedas	Sí — motivo "CANJE_RECHAZADO"
Padre entrega canje	Monedas ya descontadas, queda como definitivo	Sí — motivo "CANJE_ENTREGADO"

Auditoría completa: Cada cambio de monedas guarda saldo_anterior y saldo_posterior en historial_monedas, lo que permite reconstruir el historial completo de un niño en cualquier momento.

10. Módulo de Juegos Educativos

Extensión del sistema de gamificación: el hijo puede ganar monedas extra jugando a 5 juegos educativos en el navegador. El padre configura las monedas por juego; hay un límite de **3 partidas por juego al día**.

Arquitectura del módulo

Archivo	Responsabilidad
<code>app/Support/Juegos.php</code>	Clase estática con la constante <code>LIST</code> : array asociativo con los 5 juegos (nombre, icono, dificultad, colores de gradiente)
<code>app/Http/Controllers/Hijo/JuegoController.php</code>	index (lista de juegos + estado del día), jugar (verifica límite), completar (valida y otorga monedas)
<code>app/Http/Controllers/Padre/JuegoController.php</code>	index (muestra configuración), guardar (actualiza monedas_por_partida y activo para cada juego)
<code>resources/views/hijo/juegos/</code>	index.blade.php + 5 vistas de juego (sumas, memoria, ahorcado, ordenar, quiz)
<code>resources/views/padre/juegos/index.blade.php</code>	Panel del padre: activa/desactiva juegos y fija monedas (1–50) por partida

Tablas de BD añadidas

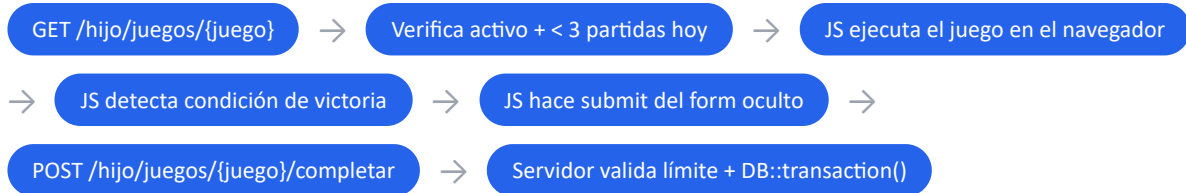
Tabla	Campos clave	Para qué sirve
<code>configuracion_juegos</code>	<code>id_padre</code> , <code>juego</code> (varchar 50), <code>monedas_por_partida</code> (default 5), <code>activo</code> (boolean)	Configuración por padre y por juego. PK compuesta única (<code>id_padre</code> , <code>juego</code>)
<code>partidas</code>	<code>id_hijo</code> , <code>juego</code> , <code>monedas_ganadas</code> , <code>created_at</code>	Registro de cada partida ganada. Se usa para comprobar el límite diario (<code>whereDate('created_at', today())</code>)

ENUM extendido: La columna `historial_monedas.motivo` era `ENUM('TAREA','ACTIVIDAD','CANJE','AJUSTE_PADRE','BONIFICACION')`. Se añadió el valor `'JUEGO'` mediante `ALTER TABLE ... MODIFY COLUMN motivo ENUM(...,'JUEGO')` en la migración.

Los 5 juegos (client-side JavaScript puro)

Clave	Nombre	Mecánica	Dificultad
<code>sumas</code>	Operaciones Matemáticas	10 sumas/restas aleatorias, hay que acertar 7 de 10	Fácil
<code>memoria</code>	Juego de Memoria	16 cartas (8 parejas de emojis) — encontrar todas las parejas	Medio
<code>ahorcado</code>	Adivina la Palabra	20 palabras en español con tildes y ñ; 6 errores permitidos; input libre con auto-envío	Medio
<code>ordenar</code>	Ordena los Números	10 fichas desordenadas, tocarlas en orden ascendente	Fácil
<code>quiz</code>	Quiz de Conocimiento	8 preguntas de cultura general, hay que acertar 6 de 8	Difícil

Flujo técnico completo de una partida



Transacción al completar partida — JuegoController@completar

```

// 1. Doble comprobación del límite (el JS podría manipularse)
$partidasHoy = Partida::where('id_hijo', $hijo->id_hijo)
                    ->where('juego', $juego)
                    ->whereDate('created_at', today())
                    ->count();
if ($partidasHoy >= 3) return redirect()->back();

// 2. Transacción atómica: o todo o nada
DB::transaction(function () use ($hijo, $config, $juego) {
    $saldoAnterior = $hijo->monedas;
    // Respetar el tope máximo de monedas del hijo
    $nuevoSaldo = min($saldoAnterior + $config->monedas_por_partida, $hijo->monedas_tope);
    $ganadas    = $nuevoSaldo - $saldoAnterior;

    $hijo->update(['monedas' => $nuevoSaldo]);

    Partida::create(['id_hijo' => $hijo->id_hijo, 'juego' => $juego, 'monedas_ganadas' => $ganadas]);

    HistorialMonedas::create([
        'id_hijo'      => $hijo->id_hijo,
        'cantidad'     => $ganadas,
        'saldo_anterior' => $saldoAnterior,
        'saldo_posterior' => $nuevoSaldo,
        'motivo'       => 'JUEGO',
    ]);
});

```

Clave de diseño: La lógica del juego está 100% en JavaScript del cliente (sin llamadas AJAX intermedias). El servidor solo interviene al inicio (¿puede jugar?) y al final (POST de victoria). Esto mantiene el backend simple y los juegos rápidos y sin latencia.

Decisión técnica: migración idempotente

MySQL **no es transaccional en DDL**: si una migración falla a mitad, la tabla queda creada físicamente pero sin registro en la tabla `migrations`. Al re-ejecutar, el `CREATE TABLE` falla con "table already exists" y el servidor no arranca. Solución: guardas `Schema::hasTable()` e inyección del `NOT NULL` en el `ALTER TABLE` del `ENUM`, envuelto en `try/catch`.

```

// Idempotente: no falla si se ejecuta dos veces
if (!Schema::hasTable('configuracion_juegos')) {
    Schema::create('configuracion_juegos', function (Blueprint $table) { ... });
}
try {
    DB::statement("ALTER TABLE historial_monedas MODIFY COLUMN motivo
        ENUM('TAREA', 'ACTIVIDAD', 'CANJE', 'AJUSTE_PADRE', 'BONIFICACION', 'JUEGO') NOT NULL");
} catch (\Throwable $e) { /* ya estaba añadido – se ignora */ }

```

11. Gamificación avanzada — Rachas, Logros y Niveles

Además del sistema básico de monedas, la app incorpora mecánicas de gamificación para aumentar la motivación: rachas de días consecutivos, logros desbloqueables y sistema de niveles. Toda la lógica está centralizada en `app/Services/AchievementService.php`.

AchievementService — patrón de uso

```
// ValidacionController – al validar una tarea:
$achievements = app(AchievementService::class)->onTareaValidada($hijo);

// JuegoController – al completar una partida:
$achievements = app(AchievementService::class)->onPartidaGanada($hijo);
```

Devuelve un array de notificaciones (nombre, icono, monedas) para mostrar al usuario.

Rachas (Streaks)

Campo en hijos	Tipo	Descripción
<code>racha_actual</code>	INT default 0	Días consecutivos con al menos 1 tarea validada
<code>racha_max</code>	INT default 0	Racha máxima histórica del hijo

Lógica: al validar una tarea se comprueba si el día anterior ya tenía una instancia validada. Si sí, `racha_actual++`; si no, se reinicia a 1. Se actualiza `racha_max` si se supera. Bonus de monedas al alcanzar 7 días (+50 monedas) y 30 días (+200 monedas), registrado con motivo `'BONIFICACION'`.

Logros

Clave	Trigger	Bonus
<code>primera_tarea</code>	1ª tarea validada	10 monedas
<code>5_tareas</code>	5 tareas validadas	25 monedas
<code>25_tareas</code>	25 tareas validadas	50 monedas
<code>100_tareas</code>	100 tareas validadas	100 monedas
<code>primera_partida</code>	1ª partida ganada	15 monedas
<code>10_partidas</code>	10 partidas ganadas	30 monedas
<code>primer_canje</code>	1er canje solicitado	20 monedas

Niveles (1–10)

```
// Hijo.php
public function nivel(): int {
    $m = $this->monedas_historicas; // nunca baja – solo sube al ganar
    if ($m >= 10000) return 10;
    if ($m >= 5000) return 9;
    if ($m >= 2500) return 8;
    // ...
    return 1;
}
```

`monedas_historicas` es un contador acumulativo en la tabla `hijos` que nunca se descuenta al canjear, a diferencia de `monedas`. Esto permite calcular el nivel en $O(1)$ sin consultar el historial completo.

Mejoras en tareas: franja horaria y categoría

Campo	Valores	Uso
<code>franja</code>	CUALQUIERA / MANANA / TARDE / NOCHE	El hijo ve sus tareas agrupadas por momento del día
<code>categoría</code>	VARCHAR libre (predefinidas: Higiene, Hogar, Deberes...)	Filtro visual, icono en la tarea

Al crear una tarea (`padre/tareas/create.blade.php`) el padre puede elegir franja y categoría mediante plantillas rápidas o manualmente.

12. Estadísticas y Calendario

El panel de estadísticas (`GET /padre/estadisticas` → `EstadisticasController@index`) ofrece al padre una visión global de toda la actividad de sus hijos.

Componentes del dashboard

Bloque	Datos	Cómo se calcula
Resumen por hijo	Tasa completado %, monedas validadas, racha, nivel, barra de progreso	Agregación Eloquent sobre <code>tarea_instancias</code>
Monedas por semana	Gráfico de barras — últimas 8 semanas	<code>YEARWEEK(fecha,1)</code> agrupado en <code>historial_monedas</code>
Juegos más jugados	TOP 5 por nº partidas + monedas ganadas	<code>GROUP BY juego ORDER BY partidas DESC LIMIT 5</code>
Partidas recientes	Últimas 20 partidas: hijo, juego, monedas, fecha	<code>Partida::with('hijo')->latest()->limit(20)</code>
Calendario del mes	Grid 7xN coloreado por estado de tareas	Ver detalle abajo

Gráfico de barras sin librerías

```
@php $maxVal = $monedasSemana->max() ?: 1; @endphp
<div style="height:{{ round(($total / $maxVal) * 100) }}px; min-height:4px"
    class="w-full rounded-t-lg bg-indigo-400"></div>
```

Las alturas se normalizan al contenedor de 128px usando el máximo como denominador. Sin Chart.js ni ninguna librería externa.

Calendario — dos bugs resueltos

Bug 1 — `grid-cols-7` en Tailwind v4: Tailwind v4 solo genera `grid-cols-1` a `grid-cols-6` en el CSS por defecto. `grid-cols-7` no aparece y el calendario se mostraba en una sola columna. Solución: usar inline style directamente: `style="display:grid;grid-template-columns:repeat(7,1fr);gap:4px"`

Bug 2 — claves `Y-m-d` nunca coinciden: `->groupBy('fecha_programada')` en Eloquent usa `Carbon::__toString()` como clave → `"2026-05-01 00:00:00"`. En la vista se buscaba con `$instanciasMes[$fecha]` donde `$fecha` era `"2026-05-01"`. Nunca coincidía → todos los días grises. Solución:

```

->selectRaw('DATE_FORMAT(fecha_programada, "%Y-%m-%d") as fecha_dia, estado, COUNT(*) as n
->groupBy('fecha_dia', 'estado')
->get()
->groupBy('fecha_dia');

```

Colores del calendario

Color	Estado	Prioridad
Verde	VALIDADA	1ª — si alguna instancia del día está validada
Ámbar	PENDIENTE / COMPLETADA	2ª — si hay pendientes pero ninguna validada
Rojo	RECHAZADA	3ª — solo rechazadas sin ninguna validada
Gris claro	Sin tareas	No hay instancias ese día

La leyenda de colores usa **inline styles** en vez de clases Tailwind (`bg-green-500`) para garantizar que se muestre independientemente de lo que Tailwind genere.

13. Despliegue en Railway

Infraestructura

Componente	Servicio Railway	Detalle
App Laravel	kids-routine	Deploy automático desde GitHub (rama main)
Base de datos	MySQL	Volumen persistente, variables de entorno inyectadas

Variables de entorno en Railway (servicio kids-routine)

Variable	Valor
APP_ENV	production
APP_KEY	base64:... (generado con artisan key:generate)
APP_URL	https://kids-routine-production.up.railway.app
DB_CONNECTION	mysql
DB_HOST / DB_PORT / DB_DATABASE / DB_USERNAME / DB_PASSWORD	Copiadas del servicio MySQL
PORT	8000

nixpacks.toml — build sin npm

```
[phases.build]
cmds = []      # ← vacío: salta npm build (assets ya pre-compilados y commiteados)

[start]
cmd = "php artisan config:cache && php artisan route:cache && php artisan migrate --force && pl
```

¿Por qué pre-compilar los assets? Vite 8 requiere Node 22+, pero Railway usa Node 20. Solución: compilar localmente con `npm run build`, commitear la carpeta `public/build/` y saltar el build en Railway con `cmds = []`.

Proceso de deploy

1. Se hace `git push` al repositorio de GitHub
2. Railway detecta el push automáticamente
3. Nixpacks instala dependencias de Composer (`vendor/`)
4. Se ejecuta el start command: caché de config → caché de rutas → migraciones → servidor
5. Railway asigna el dominio público y redirige el tráfico

14. Preguntas frecuentes en la defensa

¿Por qué Laravel y no otro framework?

Laravel es el framework PHP más usado en España y el sector. Ofrece ORM (Eloquent), sistema de migraciones, autenticación, y una comunidad enorme. Para un TFG de DAW es la elección natural en backend PHP.

¿Qué es Eloquent ORM?

Es el ORM de Laravel. Mapea tablas de BD a clases PHP. En vez de escribir SQL directamente, usas métodos: `Hijo::where('id_padre', $id)->get()`. Cada fila es un objeto PHP.

¿Qué es una migración?

Un archivo PHP que define la estructura de una tabla BD. Al ejecutar `php artisan migrate`, Laravel crea/actualiza las tablas. Permite versionar la BD igual que el código.

¿Cómo funciona el middleware?

Es un filtro que se ejecuta antes de que llegue la petición al controlador. `EsPadre` comprueba que hay sesión activa de Laravel; si no, redirige al login. `EsHijo` comprueba que hay un `hijo_id` en la sesión PHP.

¿Por qué PIN en vez de contraseña para los niños?

Los niños de 6-10 años no recuerdan contraseñas alfanuméricas. Un PIN de 4 dígitos es intuitivo, y se añade bloqueo temporal (15 min) tras 3 intentos fallidos para mantener la seguridad.

¿Qué es Tailwind CSS?

Framework CSS "utility-first". En vez de definir clases semánticas (`.boton-azul`), aplicas clases de utilidad directamente en el HTML: `class="bg-blue-500 text-white px-4 py-2 rounded"`. Más rápido para protipar, sin ficheros CSS propios.

¿Cómo se despliega automáticamente?

Railway está conectado a GitHub. Cada `git push` a la rama main dispara un nuevo deployment automático. No hay FTP ni servidores que configurar manualmente.

¿Por qué los juegos corren en el cliente (JavaScript) y no en el servidor?

Los juegos tienen lógica de interacción en tiempo real (cartas que se voltean, letras que se revelan, temporizadores) que sería imposible de gestionar con peticiones HTTP síncronas al servidor. El servidor solo interviene en dos momentos: al inicio (¿puede jugar hoy?) y al final (POST de victoria). Esta separación mantiene los juegos fluidos y el backend sin estado de juego.

¿Cómo se evita que un niño envíe el POST de victoria sin jugar realmente?

El servidor comprueba el límite de 3 partidas al día con `whereDate('created_at', today())`. Si un niño envía el POST directamente sin jugar, consumiría uno de sus 3 intentos diarios igualmente. No se valida si "de verdad ganó" — se confía en que el JS del cliente es la fuente de verdad para la condición de victoria, y el límite diario es la protección real.

¿Qué es una TareaInstancia vs una Tarea?

Tarea = plantilla (ej: "Hacer la cama, todos los días, 5 monedas"). **TareaInstancia** = ocurrencia concreta de esa tarea en un día específico, con su propio estado (pendiente/completada/validada). Una tarea genera una instancia nueva cada día/semana.

¿Por qué historial_monedas?

Para auditoría y trazabilidad. Si un niño dice "me robaron monedas", el padre puede ver exactamente cuándo y por qué subió o bajó el saldo. Buena práctica en cualquier sistema financiero, aunque sea de juguete.

¿Por qué Blade y no React o Vue?

Para este proyecto no hacía falta un frontend reactivo con estado complejo. Blade genera HTML en el servidor, sin bundle JS pesado y sin necesidad de una API REST separada. Los juegos sí usan JavaScript vanilla puro porque requieren interactividad en tiempo real, pero para el resto de la app (listados, formularios, dashboards) el renderizado en servidor es más sencillo, más rápido de desarrollar y más fácil de mantener.

¿Cómo protege la app contra ataques de seguridad?

Ataque	Protección en Kids Routine
CSRF (Cross-Site Request Forgery)	Laravel añade automáticamente un token <code>@csrf</code> en cada formulario POST. El middleware <code>VerifyCsrfToken</code> rechaza peticiones sin token válido.
XSS (Cross-Site Scripting)	Blade escapa todo lo que va entre <code>{{ }}</code> con <code>htmlspecialchars()</code> . Solo se usa <code>{!!}</code> cuando el HTML viene de código controlado (nunca de input del usuario).
SQL Injection	Eloquent usa PDO con consultas parametrizadas. Nunca se concatena input del usuario en SQL. Ej: <code>Hijo::where('id_padre', \$id)->get()</code> es seguro.
Fuerza bruta PIN	Bloqueo de 15 minutos tras 3 intentos fallidos (<code>bloqueado_hasta</code> en BD).
Acceso no autorizado	Middlewares <code>EsPadre</code> y <code>EsHijo</code> en cada ruta protegida. Un hijo no puede acceder al panel del padre y viceversa.

¿Cómo funciona el sistema de avatares/foto?

Tanto el padre como cada hijo pueden subir una foto de perfil. El archivo se guarda en `storage/app/public/avatars/` (en Railway, en disco efímero — mejora pendiente: S3). El modelo tiene dos métodos: `avatarUrl()` devuelve la URL pública si hay foto, o `null`; `avatarColor()` devuelve un gradiente CSS determinista basado en el `id_hijo % total_colores` para cuando no hay foto. En las vistas se usa siempre el patrón `@if(avatarUrl()) ... @else ... @endif`.

¿Qué es `DB::transaction()` y por qué lo usas?

Agrupar varias operaciones de BD en una transacción atómica: o se completan todas o no se aplica ninguna. En el módulo de juegos, una partida ganada implica: actualizar monedas del hijo + crear registro en `partidas` + crear registro en `historial_monedas`. Sin transacción, si falla el paso 2, el hijo tendría monedas sin registro de auditoría — inconsistencia. Con `DB::transaction()`, si cualquier paso lanza una excepción, se hace rollback automático de todo.

¿Qué es un Service en Laravel y por qué usaste uno para los logros?

Un Service es una clase PHP simple (sin herencia de Laravel) que encapsula lógica de negocio compleja que no pertenece a un único controlador. El `AchievementService` centraliza rachas, logros y niveles: si lo hubiera puesto en el controlador de validaciones, habría que duplicarlo en el de juegos y en el de canjes. Con el servicio, cualquier controlador llama a `app(AchievementService::class)->onTareaValidada($hijo)` y obtiene el resultado sin saber cómo está implementado por dentro.

¿Cómo se generan las estadísticas sin una librería de gráficos?

El gráfico de barras semanales se construye en HTML puro: cada barra es un `<div>` con altura calculada en Blade como porcentaje del máximo (`round(($total / $maxVal) * 100)px`). El calendario es un `display:grid` con 7 columnas. Sin Chart.js ni ninguna dependencia externa, lo que simplifica el bundle y el mantenimiento.

¿Has hecho tests automatizados?

No se implementaron tests unitarios ni de integración en este TFG, por limitación de tiempo y alcance. Laravel incluye PHPUnit y la clase `TestCase` para escribirlos. Las pruebas se realizaron manualmente en local con Laragon y en producción con Railway. Como mejora futura, escribiría tests para los flujos críticos: validación de tareas, canje de recompensas y límite diario de juegos.

¿Qué mejorarías si tuvieras más tiempo?

- **Almacenamiento de fotos en S3:** ahora los avatares se guardan en disco local (se pierden en cada deploy de Railway).
- **Tests automatizados** con PHPUnit para los flujos críticos.
- **Notificaciones push** cuando el padre valida una tarea (usando Laravel Notifications).
- **App móvil** con una API REST + React Native o Flutter, ya que el target principal son niños con móvil/tablet.
- **Más juegos** educativos (los 5 actuales son un buen punto de partida).

¿Qué problemas encontraste en el desarrollo?

- **Nombres de tablas no estándar:** Las tablas están en español, Laravel espera inglés en plural. Solución: definir `$table` y `$primaryKey` en cada modelo.
- **Compatibilidad Node.js en Railway:** Vite 8 requiere Node 22+, Railway tenía Node 20. Solución: pre-compilar assets localmente y saltarse el build remoto.
- **PIN vs Auth de Laravel:** Laravel Auth está pensado para email+contraseña. El PIN del hijo usa `Hash::check()` manualmente con gestión de bloqueo propia.
- **Migración DDL en Railway:** MySQL no es transaccional en DDL; una migración fallida a mitad dejaba la tabla creada físicamente pero sin registro en `migrations`. Al reintentar el deploy fallaba con "table already exists". Solución: `Schema::hasTable()` + `try/catch`.
- **Emoji 🪙 no soportado en dispositivos antiguos:** Unicode 13.0 no disponible en algunos sistemas. Solución: Blade component `<x-moneda />` con SVG inline.
- **Tailwind v4 no genera `grid-cols-7`:** Tailwind CSS v4 solo incluye en el CSS generado las clases que detecta en los archivos. `grid-cols-7` no estaba en ningún archivo de referencia y no se generaba, rompiendo el calendario. Solución: inline style `display:grid;grid-template-columns:repeat(7,1fr)`.
- **Claves del calendario no coincidían:** `->groupBy('fecha_programada')` en Eloquent usa `Carbon::__toString()` como clave de la colección, que incluye la hora (`"2026-05-01 00:00:00"`). En la vista se buscaba por `"2026-05-01"` → nunca coincidía → todos los días grises. Solución: `DATE_FORMAT(fecha_programada, "%Y-%m-%d")` en el `selectRaw`.
- **ENUM con carácter Ñ en MySQL:** El valor `'MAÑANA'` en un ENUM causaba errores de charset en Railway (utf8mb3 vs utf8mb4). Solución: renombrar el valor a `'MANANA'` (sin tilde) en la migración, el modelo y las vistas.

15. Datos de demo para la presentación

URL: <https://kids-routine-production.up.railway.app>

Rol	Credenciales
Padre	Email: <code>demo@kidsroutine.com</code> · Contraseña: <code>demo123</code>
Sofía (8 años)	PIN: <code>1234</code> · 30 monedas · Tareas: hacer la cama, estudiar, ordenar habitación
Pablo (6 años)	PIN: <code>5678</code> · 15 monedas · Tareas: lavarse los dientes, recoger juguetes

Flujo para demostrar en la defensa

1. Abrir la URL → mostrar landing page pública

2. Login como padre demo (demo@kidsroutine.com / demo123)
3. Mostrar panel padre: dashboard con resumen, hijos, tareas, recompensas
4. Mostrar **Juegos** → **configuración**: cambiar monedas de un juego, activar/desactivar
5. Entrar como hijo: Modo Hijo → Sofía → PIN 1234
6. Mostrar panel hijo: tareas del día con monedas actuales
7. Marcar una tarea como completada
8. Ir a **Juegos** → jugar al Quiz o Sumas → ganar partida → ver monedas sumadas
9. Ir a **Tienda** → canjear una recompensa
10. Volver como padre → **Validaciones** → Validar la tarea completada
11. Como padre → **Canjes** → Aprobar → Entregar

Consejo: Muestra primero la configuración de juegos desde el padre (paso 4) antes de entrar como hijo, así el tribunal ve la relación padre-configura / hijo-juega.